UCLM
UNIVERSIDAD DE CASTILLA-LA MANCHA

Sesión 4
Comandos avanzados

Escuela Superior de Informática
Universidad de Castilla-La Mancha

Contenido

- Utilidades de Terminal
- Tareas

- Entrada/Salida Estándar
- Pipes y Redirección

Utilidades de Terminal

- grep
- find
- Variables de entorno

grep - Global Regular Expression Print

- Búsqueda global en función de expresiones regulares

http://osr507doc.sco.com/en/OSTut/graphics/grep.gif

- Uso

```
grep [opciones] [patrón] [ficheros]
```

grep. Opciones

`-c,--count:` Suprime la salida normal; en su lugar muestra el número de líneas que concuerdan con el patrón para cada fichero de entrada.

`-e patrón,--regexp=PATRÓN:` Emplea patrón como el patrón; útil para proteger patrones que comiencen con -.

`-f fichero,--file=FICHERO:` Obtiene el patrón de fichero.

`-i,--ignore-case:` No hace caso de si las letras son mayúsculas o minúsculas ni en el patrón ni en los ficheros de entrada.

grep. Opciones

`-v,--revert-match:` Selecciona las líneas que NO coninciden con el criterio de búsqueda.

`-w,--word-regexp:` Selecciona solo las líneas que contienen concordancias que forman palabras completas. La cadena debe estar al principio o la final de la línea, y debe estar precedida y seguida por un carácter que no forme parte de una palabra. Los caracteres que se consideran como parte de palabras son letras, dígitos y el subrayado.

`-s,--silent:` Suprime los mensajes de error sobre ficheros que no existen o no se pueden leer.

UCLM UNIVERSIDAD DE CASTILLA-LA MANCHA

Meta-carácteres y Expresiones Regulares

- **Meta-carácteres**

  * cero o más letras

  ? un solo carácter

  [ ] lista de carácteres posibles

  - rango de valores

  \ ignorar el meta-carácter siguiente

- **Expresiones Regulares**

  ^ al principio de la línea

  $ al final de la línea

  [ ] lista de carácteres posibles

  - rango de valores

  **** ignorar el meta-carácter siguiente

  [^expr] cualquier expresión excepto *expr*

  r* cero o más ocurrencias de r

grep. Opciones básicas

- Búsqueda de la ocurrencia de una palabra en un fichero.

```
$ grep palabra fichero
```

- Sin distinción entre mayúsculas y minúsculas.

```
$ grep -i palabra fichero
```

- Búsqueda recursiva sobre todos los ficheros de un directorio.

```
$ grep -r palabra ruta
```

rgrep es recursivo por defecto

- Cuenta el número de ocurrencias de una palabra en un determinado fichero.

```
$ grep -c palabra fichero
```

grep. Opciones básicas

- Busca las líneas que no contengan una determinada palabra.

```
$ grep -v palabra fichero
```

- Lista el nombre de aquellos archivos, de un determinado tipo, que contengan una cadena.

```
$ grep -l palabra *.ext
```

```
$ grep -l "main()" *.c
```

- Para que la búsqueda resulte más visual, se puede utilizar un color para resaltar el patrón.

```
$ grep --color palabra fichero
```

grep. Expr. regulares

- Búsqueda de palabras completas, evitando la aparición en subcadenas.

```
$ grep -w palabra fichero
```

- Mostrar las líneas que comienzan con una secuencia de caracteres determinado.

```
$ grep '^seq' fichero
```

- Búsqueda de líneas que contengan alguna de las palabras.

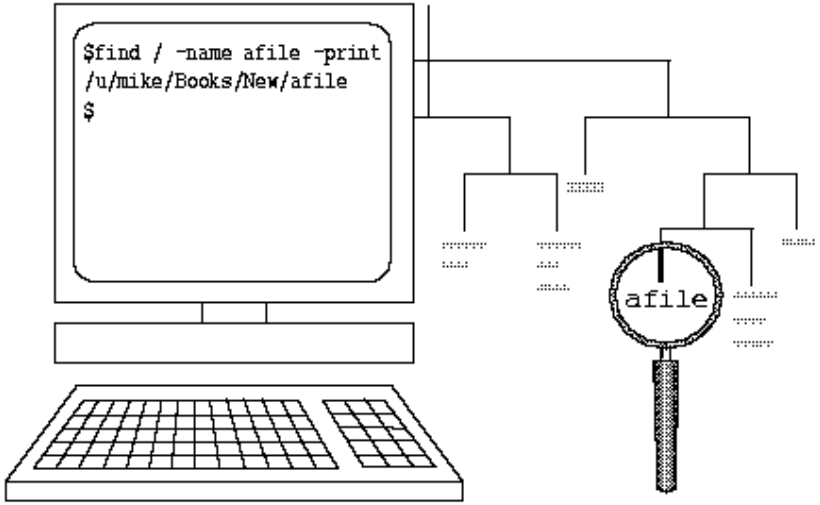
```
$ grep -e '(word1|word2)' fichero
```

egrep usa RE por defecto

find

- Búsqueda de archivos en una jerarquía de directorios
- Uso

```
find [ruta] [expresión]
```



http://osr507doc.sco.com/en/OSTut/graphics/find.gif

find. Principales Opciones

-name <*expresion*>: especifica el patrón del nombre de archivo a buscar.

-type: especifica qué tipo de fichero se quiere buscar. Los tipos son:

- **d -** directorios
- **f -** ficheros
- **l -** enlaces simbólicos

-size +/-<*n*>: busca ficheros de un determinado tamaño (n en KiB).

-exec <*comando*> {};: Comando a ejecutar con los ficheros resultados de la búsqueda. Los {} se sustituyen por los ficheros encontrados

find. Ejemplos de uso

- Listado de todos los ficheros con extensión **.c** que cuelgan de un determinado directorio.

```
$ find [ruta] -name "*.c"
```

- Lista los ficheros que cuelguen del home de usuario "**alumno**" y contengan la subcadena "**practica**"

```
$ find /home/alumno -name "*practica*"
```

- Busca y compila los ficheros con extensión **.c**

```
$ find /home/alumno/practica1 -name "*.c" -exec gcc {} \;
```

- Búsqueda de aquellos ficheros de más de 500MiB.

```
$ find /home/alumno -size +500M
```

Variables de entorno

- Las variables de entorno se utilizan para proporcionar valores de configuración a los programas.
- El CLASSPATH de java es un ejemplo de variable de entorno.

Variables de entorno

- El comando **`env`** muestra la lista de variables de entorno definidas en nuestra shell.
- El comando **`set`**, también las muestra, aunque se utiliza más para definirlas.
- El comando **`unset`** se utiliza elimina una variable.
- El comando **`export`**, si no está definida la variable, la define; si lo está, sustituye su valor.
- El comando **`echo`** muestra el contenido de una variable.

Variables de entorno. Ejemplos

- Muestra el contenido de la variable **PATH**.

```
$ echo $PATH
```

- Crea un archivo ls y escribe una línea de texto.

```
$ echo “echo \”hola mundo\”” > ls
```

- Dale permisos de ejecución.

```
$ chmod +x ls
```

- Modifica la variable **PATH** para que busque primero en el directorio de trabajo.

```
$ export PATH=/home/alumno:$PATH
```

- Ejecuta “**ls**” ¿qué ocurre?

Planificando Tareas

- Las tareas de administración suelen consistir en tareas rutinarias.
- Sistemas para la construcción de "*agendas de tareas*":
  - **`cron`**
  - **`at`**

http://www.flickr.com/photos/81828336@N00/2914061212/

cron

- Establece la programación de una lista de tareas a ejecutar.
- Las tareas están configuradas como entradas del fichero **`/etc/crontab`**:
  - Minutos y hora en que se lanzará la tarea.
  - Día del mes.
  - Mes.
  - Día de la semana.
  - Tarea a ejecutar (puede ser un directorio donde se encuentren las tareas a ejecutar).

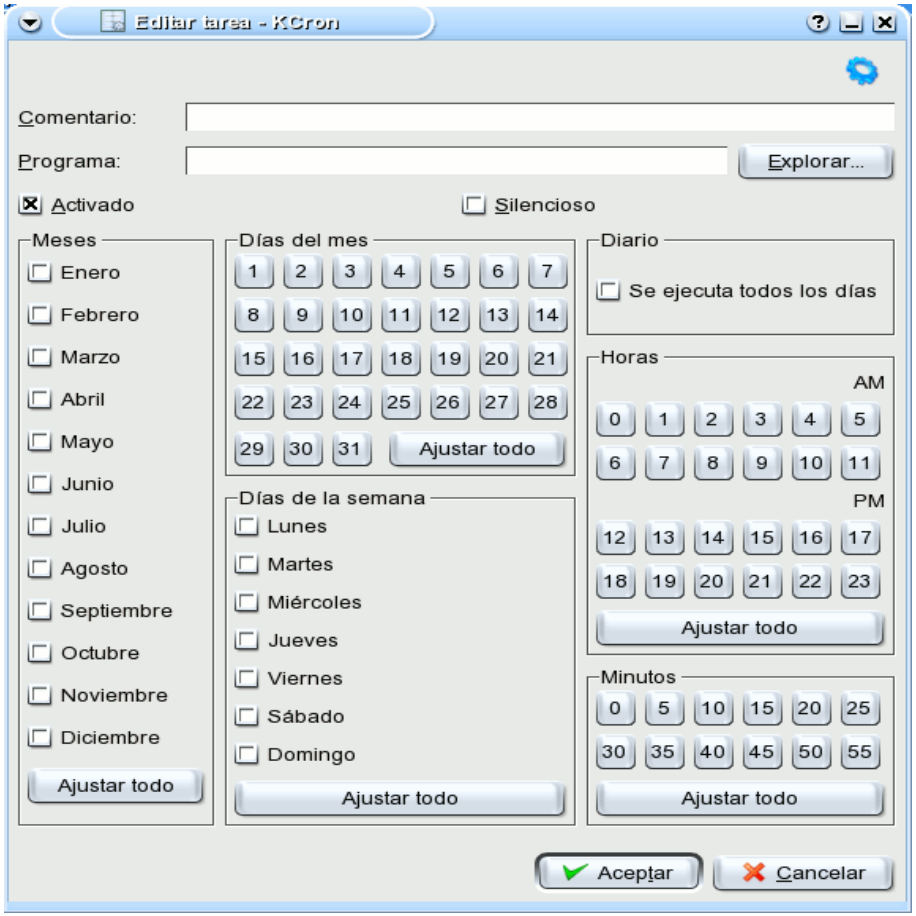
cron. Ejemplos

- Crea una copia de seguridad tar.gz del contenido de tu directorio de trabajo cada domingo de la semana.
- Programa un despertador de lunes a viernes a las 7.30 de la mañana.
- Programa una tarea que avise cuando queden 30 minutos de clase (13.30h).

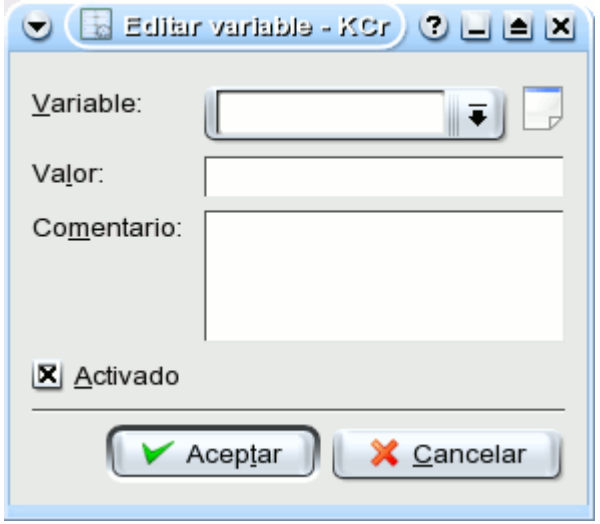
```
00 5  7 *  *    tar cfz /tmp/backup.tar.gz /home/alumno/backup
30 7  * *  1-5  DISPLAY=":0" totem /home/alumno/musica/Amelie/*.mp3
30 13 * *  *    echo "quedan 30 minutos de clase"
```

kcron

- Interfaz gráfica para cron.



- También para las variables de entorno.



at

- Permite programar una tarea, una vez, para una fecha y hora concreta.
- La fecha admite "*today*" y "*tomorrow*".
- Por defecto sólo ejecuta aplicaciones de consola. **`DISPLAY=:0`** permite lanzar aplicaciones gráficas.
- El resultado de la ejecución de la tarea se puede redirigir a un archivo para conocer el resultado.

at. Ejemplos de uso

- Reproducir una lista de canciones dentro de 10 minutos.

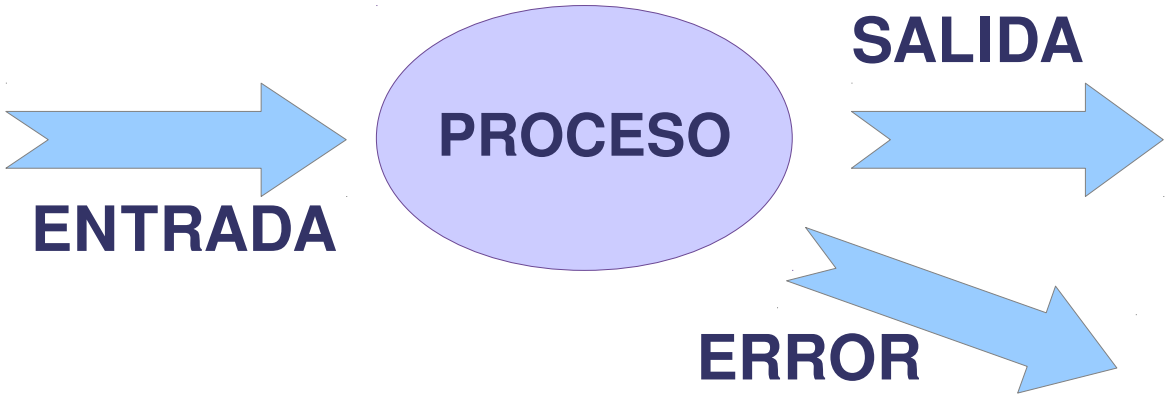
```
$ at now +10 minutes
at> DISPLAY=:0 totem /home/alumno/musica/Disc1/*.mp3
at> (Ctrl+D) <EOT>
job 17 at Thu Dec 10 10:51:00 2010
```

- Apaga el ordenador a las 14:00 de hoy.

```
$ at 14:00
at> halt
at> (Ctrl+D) <EOT>
```

E/S Estándar

- En la ejecución de un proceso (un comando por ejemplo) se utilizan tres «ficheros»:



- El intérprete de comandos configura el sistema de forma que la entrada estándar es el teclado y la salida la pantalla (ej. comandos cat y sort).

E/S Estándar

- La entrada estándar (o *stdin*), por defecto se refiere al teclado. STDIN se identifica con el descriptor 0.
- La salida estándar (o stdout), por defecto se refiere a la consola. STDOUT se identifica con el descriptor 1.
- La salida de error estándar (o stder), que de manera predeterminada se refiere a la consola, se identifica con el descriptor 2.

Redirecciones

- El intérprete de comandos nos permite redireccionar la salida estándar a un fichero.
- Redireccionar la impresión del mensaje “Hola, Mundo”, impreso dentro de 3 minutos al fichero **`/tmp/hola`**.

```
$ at now +3 minutes
at> echo “Hola, Mundo” > /tmp/hola
at> (^D) <EOT>
$ cat /tmp/hola
```

Tipos de redirección

- Redireccionamiento destructivo (si el fichero ya existe, lo sobre-escribe) utilizando >
- Redireccionamiento no destructivo (se añade al final del archivo) utilizando >>
- Redireccionamiento de la salida de error, utilizando 2>

Redireccionar la salida de error

- Ejecuta el comando grep de manera incorrecta (por ejemplo con una opción que no esté definida) y redirecciona la salida de error a un fichero.

```
$ grep -k 2> error.txt
$ cat error.txt
grep: opción inválida -- k
Modo de empleo: grep [OPCIÓN]... PATRÓN [FICHERO]...
Pruebe `grep --help' para más información.
```

Redirecciones. Ejemplos

- Crea la lista de la compra con los siguientes elementos:

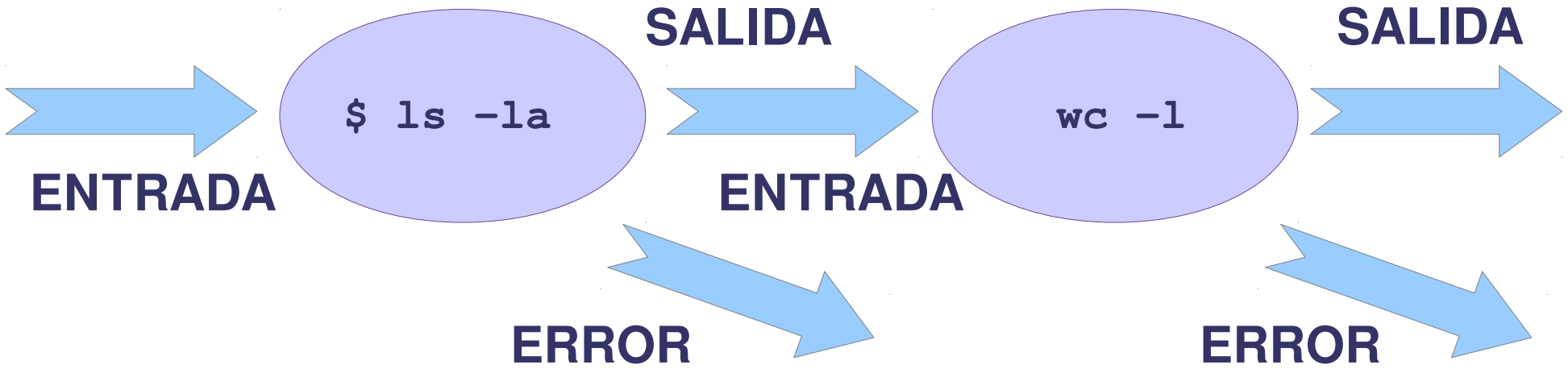
```
$ cat > lista_compra
plátanos
manzanas
leche
pan <Ctrl-D>
```

- Ordena la lista y salva la nueva como lista_ordenada:

```
$ sort < lista_compra > lista_ordenada
```

Tuberías (pipelines)

- Son un mecanismo de comunicación que permite enlazar la salida de un comando con la entrada de otro.



Tuberías. Ejemplos

- Contar el número de archivos de un directorio.

```
$ ls | wc -l
```

- Ordenar alfabéticamente (inverso).

```
$ ls | sort -r
```

- Archivos cuyo nombre comienza por “lis”.

```
$ ls | grep ^lis
```

Caso práctico

Tenemos una colección de archivos de música (en formato mp3), organizados en directorios en una estructura como la siguiente:

GRUPO - Album

01 -Nombre del tema.mp3
02 -Nombre del tema.mp3
.
.
.
12 -Nombre del tema.mp3